



Kingdom of Saudi Arabia

King Faisal University

College of Computer Sciences & Information Technology

Project Name:

Real-Time Sign Language Recognition

Full ASL Alphabet (A–Z) Using MediaPipe, SVM, Random Forest, and KNN

Subject:

Machine Learning for Computer Vision

Members

Deem Al-Subaie

Supervised by

Dr. Mohammed Shujaa

M.Sc. Artificial Intelligence Program

May 2026

Table of Contents

<i>Abstract</i>	2
1. <i>Introduction</i>	3
1.1 <i>Motivation</i>	3
1.2 <i>Problem Statement</i>	3
1.3 <i>Project Phases</i>	3
1.4 <i>Objectives</i>	3
2. <i>Related Work</i>	4
2.1 <i>Traditional Approaches</i>	4
2.2 <i>Deep Learning Approaches</i>	4
2.3 <i>Landmark-Based Approaches</i>	4
2.4 <i>Full-Alphabet ASL Recognition</i>	4
3. <i>System Architecture</i>	5
3.1 <i>Feature Representation</i>	5
3.2 <i>Classifier</i>	5
3.3 <i>Inference Engine</i>	5
4. <i>Methodology</i>	6
4.1 <i>Phase 1 — Proof of Concept</i>	6
4.2 <i>Phase 2 — Full ASL Alphabet</i>	6
4.3 <i>Data Collection</i>	6
4.4 <i>Data Augmentation</i>	6
4.5 <i>Model Training</i>	7
5. <i>Dataset and Training Results</i>	7
5.1 <i>Dataset Scale Comparison</i>	7
5.2 <i>Class Distribution</i>	7
5.3 <i>Model Hyperparameters</i>	8
6. <i>Word Builder Application</i>	8
6.1 <i>Letter Addition Logic</i>	8
6.2 <i>Keyboard Controls</i>	9
6.3 <i>Live Demonstration</i>	9
7. <i>Discussion</i>	9
7.1 <i>Strengths</i>	9
7.2 <i>Limitations</i>	10
8. <i>Conclusion and Future Work</i>	10
8.1 <i>Conclusion</i>	10
8.2 <i>Future Work</i>	10
<i>References</i>	11

Abstract

This report presents the design and implementation of a real-time sign language recognition system covering the complete American Sign Language (ASL) alphabet — 28 classes in total (A–Z plus delete and space). The system leverages Google’s MediaPipe framework for 3D hand landmark extraction and machine learning classifiers trained via scikit-learn for gesture classification. Three classifiers were evaluated: Random Forest, Support Vector Machine (SVM), and K-Nearest Neighbors (KNN). The pipeline consists of four sequential stages: data collection, augmentation to 83,076 samples across 28 classes using six geometric transformation techniques, model training with 3-fold cross-validation, and real-time inference with an interactive word-building interface. The best-performing model — SVM with RBF kernel — achieved 99.87% cross-validation accuracy and 99.55% test accuracy on the full 28-class dataset. The system was successfully deployed and demonstrated live word construction, confirming practical usability.

Keywords: Sign Language Recognition, ASL Alphabet, Hand Gesture Classification, MediaPipe, SVM, Random Forest, KNN, Computer Vision, Real-Time Inference, Data Augmentation.

1. Introduction

1.1 Motivation

Sign language is the primary mode of communication for millions of individuals in the deaf and hard-of-hearing community worldwide. Despite its importance, communication barriers persist due to the limited number of sign language interpreters. Automated sign language recognition (SLR) systems offer a promising solution by enabling real-time translation of hand gestures into text.

1.2 Problem Statement

This project addresses the challenge of building an end-to-end, real-time sign language recognition system that: (1) accurately classifies static hand gestures for all 28 classes, (2) operates in real time using a standard webcam, (3) scales robustly from a small dataset through augmentation, and (4) provides a practical word-building interface enabling users to compose words letter by letter.

1.3 Project Phases

The project was developed in two phases. Phase 1 served as a proof of concept targeting 5 signs (A–E) with 200 samples each to validate the full pipeline, achieving 100% classification accuracy. Phase 2 scaled to the complete ASL alphabet plus del and space — 28 classes — with 83,076 augmented training samples.

1.4 Objectives

- Collect a balanced webcam dataset covering all 28 classes (A–Z, del, space).
- Apply six data augmentation techniques to expand the dataset to 83,076 samples.
- Train and evaluate three classifiers (Random Forest, SVM, KNN) using 3-fold cross-validation and select the best model.
- Deploy a real-time recognition system with a word-builder interface.

2. Related Work

2.1 Traditional Approaches

Early sign language recognition systems relied on wearable data gloves. Image-based methods using color segmentation and contour analysis followed but were highly sensitive to lighting conditions and background clutter.

2.2 Deep Learning Approaches

Convolutional Neural Networks (CNNs) significantly advanced the field by operating on raw image patches. These systems achieved high accuracy but required large labeled datasets and substantial computational resources.

2.3 Landmark-Based Approaches

Google's MediaPipe Hands provides reliable real-time 3D hand landmark coordinates on CPU. The resulting compact 63-dimensional feature vectors ($21 \text{ landmarks} \times 3 \text{ coordinates}$, normalized relative to the wrist) enable lightweight classifiers to achieve strong performance with modest training data.

2.4 Full-Alphabet ASL Recognition

Recognizing all 26 ASL letters introduces specific challenges. Several letter pairs are visually similar in landmark space (notably A/S/E, U/V, M/N), requiring sufficient training data and feature richness to separate reliably.

3. System Architecture

The system is structured as a four-stage modular pipeline. Each stage produces artifacts consumed by the next, ensuring reproducibility and easy component replacement.

Stage	Module	Input	Output
1	1_Data_Collector.py	Webcam frames	raw_data.csv
2	2_Augment_Data.py	raw_data.csv	augmented_data.csv (83,076 samples, 28 classes)
3	3_Train_v2.py	augmented_data.csv	sign_model.pkl (28-class SVM)
4	4 Real Time v2.py	sign_model.pkl + Webcam	Predicted letters + Word output

Table 1: System pipeline stages.

3.1 Feature Representation

The input to the classifier is a 63-dimensional vector derived from MediaPipe's 21 hand landmarks ($21 \times 3 = 63$ values for x, y, z coordinates). All coordinates are computed relative to the wrist landmark (index 0), providing translation invariance.

3.2 Classifier

Three classifiers were trained and compared. The best-performing model is a Support Vector Machine (SVM) with RBF kernel ($C=10$, $\text{gamma}=\text{'scale'}$) — a maximum-margin classifier that maps features into a high-dimensional space to find the optimal decision boundary. Features are standardized using a StandardScaler before training. Both the scaler and trained model are serialized together into sign_model.pkl for seamless deployment.

3.3 Inference Engine

During real-time inference, predictions are accumulated in a rolling buffer of 15–20 frames, and the majority-vote prediction is taken as the current sign. A letter is committed to the word only when confidence exceeds a threshold and the prediction has been stable for a set duration.

4. Methodology

4.1 Phase 1 — Proof of Concept (5 Signs)

The project began with a proof-of-concept phase targeting five signs (A–E). In this phase, 200 samples per sign were collected via webcam, augmented to 500 per sign (2,500 total), and used to train the classifier. This phase achieved 100% test accuracy, confirming the viability of the pipeline before scaling.

4.2 Phase 2 — Full ASL Alphabet (28 Classes)

Building on the validated pipeline, the project was extended to all 28 classes (A–Z, del, space). The data collection, augmentation, and training scripts were updated accordingly.

4.3 Data Collection

The data collection module captured MediaPipe landmarks frame by frame across all 28 classes. The final augmented dataset contains 83,076 samples with 63 features each (21 landmarks $\times$ 3 coordinates).

4.4 Data Augmentation

The augmentation module expanded the dataset to 83,076 augmented samples across 28 classes using six transformation techniques:

Technique	Description	Parameters
Gaussian Noise	Adds random perturbations to all landmarks	std = 0.005
Scale	Multiplies coordinates by random scale factor	range = [0.85, 1.15]
Mirror (Flip)	Negates X coordinate, simulates left-hand view	—
Rotation	2D rotation in XY plane around wrist point	max = ± 15 deg
Translation	Uniform XY offset added to all landmarks	max = ± 0.03
Combined	Random subset of the above applied sequentially	Random mix

Table 2: Six augmentation techniques applied to all 28 sign classes.

4.5 Model Training

The training script reads augmented_data.csv (83,076 samples, 28 classes), applies StandardScaler normalization, and trains three classifiers: Random Forest (50 estimators, max_depth=10), SVM (RBF kernel, C=10), and KNN (k=5). A 3-fold cross-validation is performed for each model, and the best-performing model (SVM) is saved to sign_model.pkl.

5. Dataset and Training Results

5.1 Dataset Scale Comparison

Metric	Phase 1 (A–E)	Phase 2 (28 classes)
Signs covered	5	28 (A–Z + del + space)
Total augmented samples	2,500	83,076
Feature dimensions	63	63
Model	SVM (Best)	SVM (Best)
CV Accuracy	100.00%	99.87%
Test Accuracy	100.00%	99.55%

Table 3: Dataset and results — Phase 1 vs Phase 2.

5.2 Class Distribution

After augmentation, the dataset contains 83,076 total samples across 28 classes. The distribution is approximately balanced, with each class containing between 2,079 (space) and 3,279 (S) samples.

Sign	Samples	Sign	Samples	Sign	Samples	Sign	Samples
A	3,030	H	3,228	O	3,261	V	2,979
B	2,976	I	3,150	P	3,273	W	3,042
C	3,123	J	3,009	Q	3,192	X	3,111

D	3,186	K	3,114	R	3,177	Y	3,270
E	3,021	L	2,958	S	3,279	Z	2,883
F	3,225	M	1,830	T	3,045	del	2,274
G	3,192	N	2,013	U	3,156	space	2,079

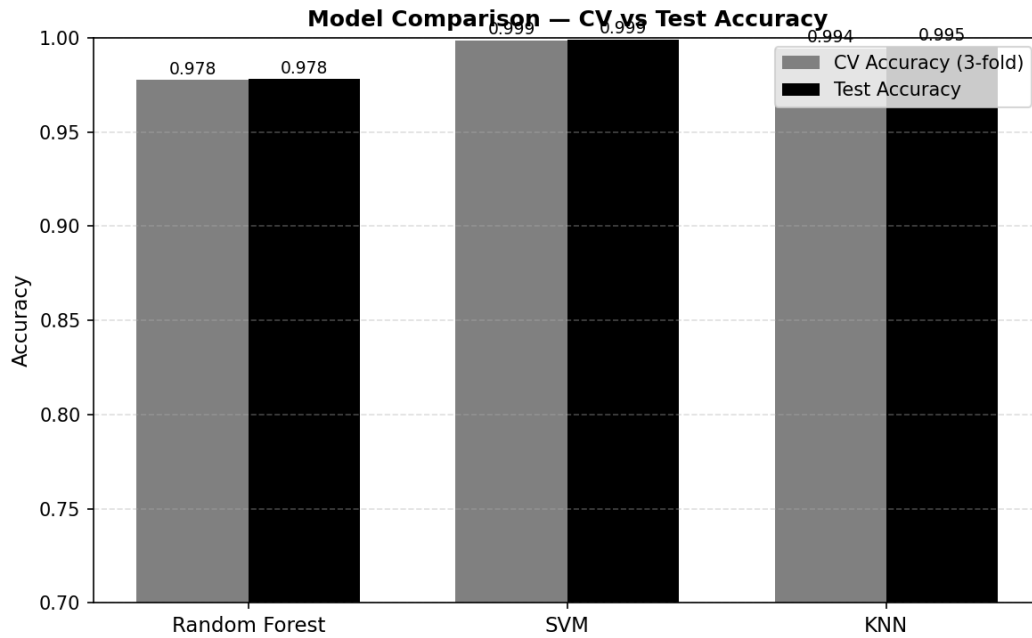
Table 4: Augmented sample distribution across all 28 classes.

5.3 Model Comparison

Three classifiers were trained and evaluated using 3-fold cross-validation on the same 83,076-sample dataset. SVM with RBF kernel achieved the highest accuracy and was selected as the deployed model.

Model	CV Accuracy (3-fold)	Test Accuracy	Selected
Random Forest	97.79% ($\pm 0.13\%$)	97.80%	No
SVM (RBF)	99.87% ($\pm 0.04\%$)	99.55%	✓ Yes
KNN (k=5)	99.45% ($\pm 0.05\%$)	99.45%	No

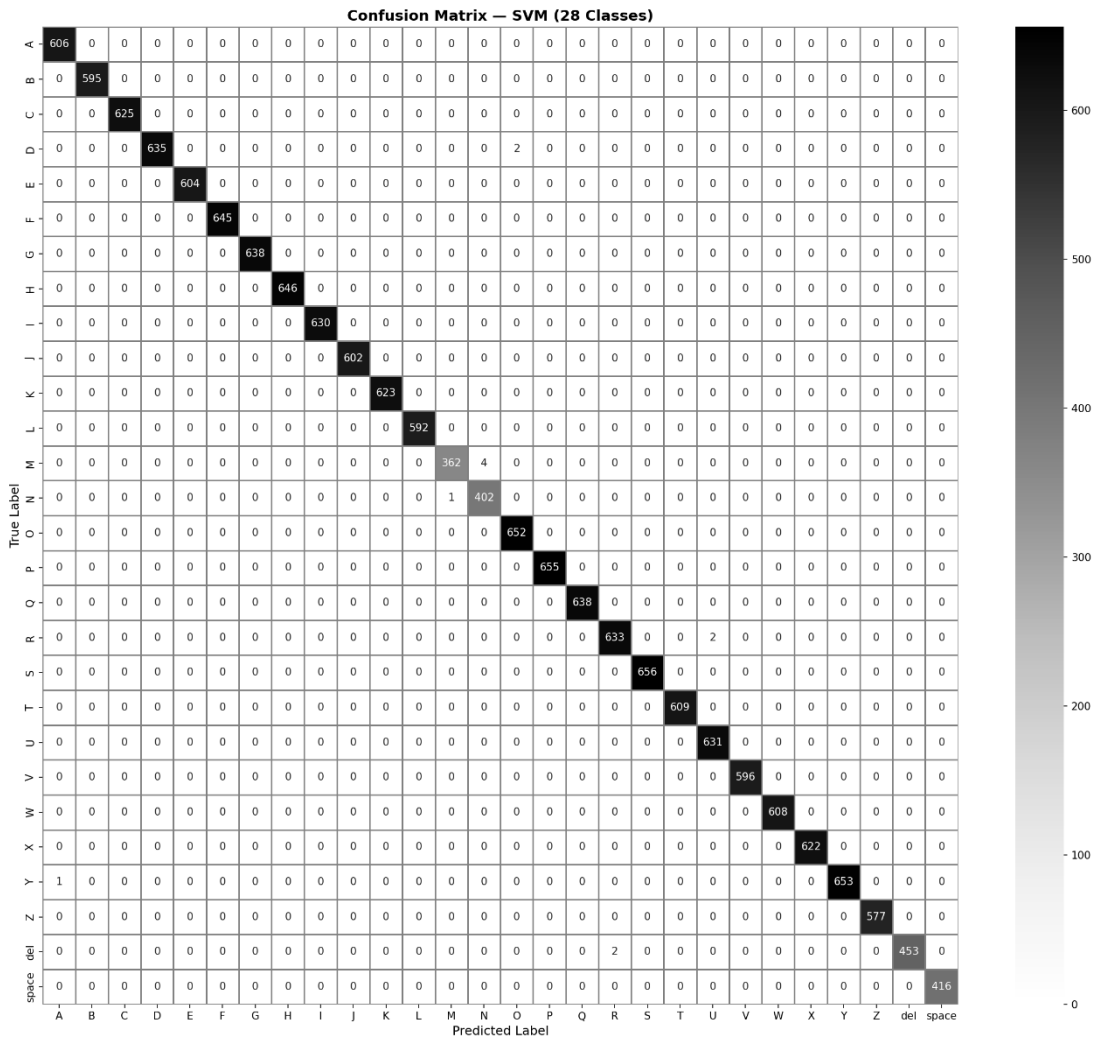
Table 5: Three-model comparison on 28-class ASL dataset (3-fold CV).



5.4 Model Hyperparameters

Hyperparameter	Value
Algorithm	SVM (RBF kernel)
Kernel	RBF
C (regularization)	10
Feature scaling	StandardScaler (zero mean, unit variance)
Input features	63 (21 landmarks $\times$ 3 coordinates, wrist-relative)
Output classes	28 (A–Z + del + space)
Validation strategy	3-fold cross-validation
CV Accuracy	99.87%
Test Accuracy	99.55%
Framework	scikit-learn

Table 5: Best model (SVM) configuration and results for the 28-class problem.



6. Word Builder Application

A core deliverable of this project is the word-building interface, which transforms the sign classifier into a practical communication tool. With the full A–Z model, users can spell any English word by signing individual letters in sequence.

6.1 Letter Addition Logic

A letter is committed to the current word only when: (1) hand confidence is at or above the configured threshold (0.65–0.70), (2) the same sign has been stable for the required duration (1.2 seconds or 20 frames), and (3) the same letter has not been added within the 3-second cooldown period.

6.2 Keyboard Controls

Key	Action
SPACE	Insert space character
B	Backspace — delete last character
C	Clear entire current word
S	Save word to history
ESC / Q	Quit the application

Table 6: Keyboard controls for the word builder application.

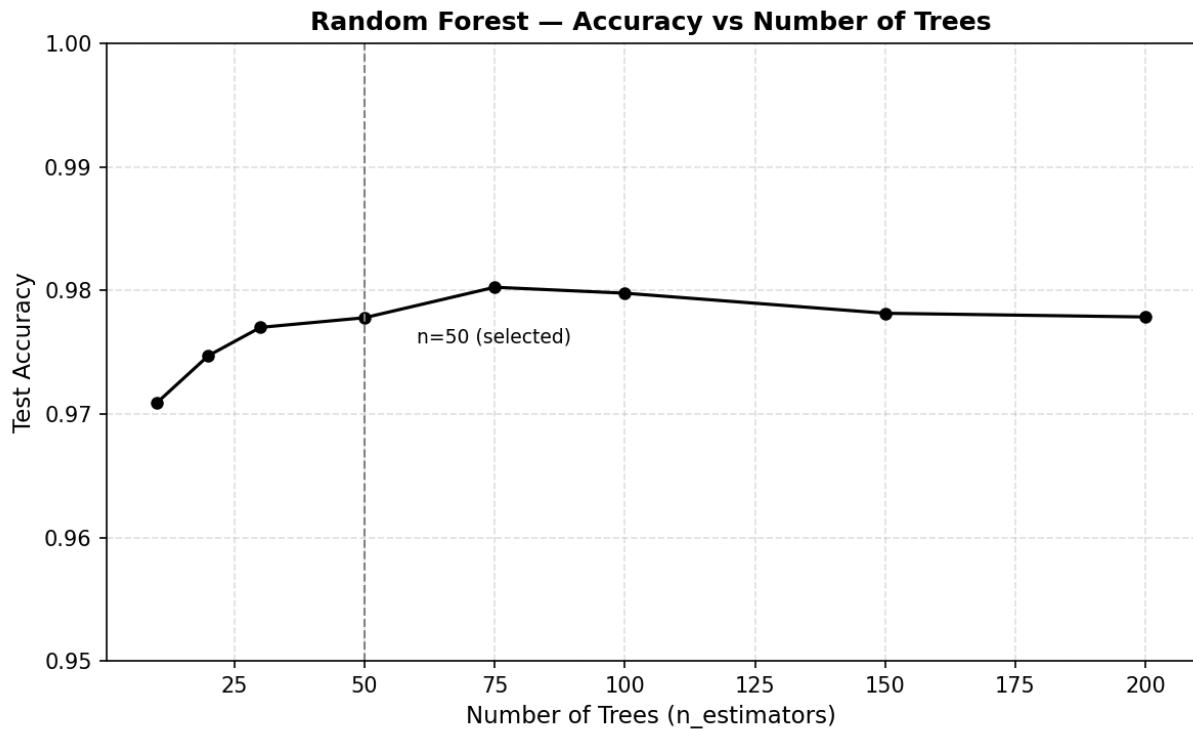
6.3 Live Demonstration

During testing, the word builder successfully recognized and composed the word 'BAD' by signing letters B, A, and D in sequence. The application ran stably on Python 3.10 with mediapipe 0.10, displaying real-time confidence and stability feedback throughout the session.

7. Discussion

7.1 Strengths

- Full alphabet coverage — supports all 28 classes, enabling real word construction.
- Lightweight and portable — runs entirely on CPU using a standard webcam; no GPU required.
- Modular pipeline — each of the four stages can be updated or replaced independently.
- High accuracy — SVM achieved 99.55% test accuracy on 28-class classification, outperforming Random Forest (97.80%) and KNN (99.45%).



7.2 Limitations

- Static signs only — motion-based ASL letters (J and Z require movement) cannot be recognized.
- Single-user data — data collected by one user; cross-user generalization requires multi-user datasets.
- Similar sign pairs — letter pairs (A/S/E, U/V, M/N) are visually similar and may cause confusion.
- Version sensitivity — MediaPipe API breaking changes between versions required code updates.

8. Conclusion and Future Work

8.1 Conclusion

This project successfully delivered a complete end-to-end real-time sign language recognition system covering 28 classes (A–Z, del, space). The system was developed in two phases: a proof-of-concept on 5 signs achieving 100% test accuracy, followed by a full-scale extension with an 83,076-sample augmented training dataset. Three classifiers were evaluated — Random Forest (97.79% CV), KNN (99.45% CV), and SVM (99.87% CV). The best model, SVM with RBF kernel, achieved 99.55% test accuracy, confirming the viability of landmark-based machine learning for full-alphabet ASL fingerspelling recognition.

8.2 Future Work

- Dynamic sign support — incorporate LSTM or Transformer models for motion-based signs (J, Z).
- Multi-user robustness — collect data from multiple users to improve cross-user generalization.
- Text-to-speech integration — connect the word builder output to a TTS engine for audio output.
- Mobile deployment — convert the model to TFLite or ONNX for deployment on mobile devices.

References.

- [1] Zhang, F. et al. (2020). MediaPipe Hands: On-device Real-time Hand Tracking. arXiv:2006.10214.
- [2] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.
- [3] Bradski, G. (2000). The OpenCV Library. Dr. Dobb's Journal of Software Tools.
- [4] Lugaresi, C. et al. (2019). MediaPipe: A Framework for Perceiving and Processing Reality. CVPR Workshop.
- [5] Wadhawan, A., & Kumar, P. (2021). Sign Language Recognition Systems: A Decade Systematic Literature Review. Archives of Computational Methods in Engineering, 28, 785–813.
- [6] Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.